Lab 08: Animasyon ve Zamanlayıcılar

PyGame ile Oyun Geliştirme – Hafta 8

Ad Soyad: _____	Numara: _____	Tarih: _____

1 Çalışma 1: Sprite Sheet'ten Frame Kesme ve Basit Animasyon

Örnek Kod

Aşağıdaki programı `frame_animasyon.py` dosyasına yazın ve çalıştırın:

```python
import pygame, math

pygame.init()
ekran = pygame.display.set_mode((600, 400))
pygame.display.set_caption("Sprite Sheet Animasyon")
saat = pygame.time.Clock()

# Programatik sprite sheet oluştur (4 frame, 64x64 piksel)
FRAME_W, FRAME_H = 64, 64
FRAME_SAYISI = 4
sheet = pygame.Surface((FRAME_W * FRAME_SAYISI, FRAME_H))
sheet.fill((0, 0, 0))
sheet.set_colorkey((0, 0, 0))

for i in range(FRAME_SAYISI):
    x = i * FRAME_W
    # Her frame'de farkli boyutta daire (nabiz efekti)
    yaricap = 15 + i * 5
    pygame.draw.circle(sheet, (50, 200, 255), (x + 32, 32), yaricap)
    # Goz pozisyonlari frame'e gore degisir
    goz_x = 22 + i * 2
    pygame.draw.circle(sheet, (255, 255, 255), (x + goz_x, 24), 5)
    pygame.draw.circle(sheet, (255, 255, 255), (x + goz_x + 18, 24), 5)
    pygame.draw.circle(sheet, (0, 0, 100), (x + goz_x + 1, 24), 2)
    pygame.draw.circle(sheet, (0, 0, 100), (x + goz_x + 19, 24), 2)

# subsurface ile frame'leri kes
frameler = []
for i in range(FRAME_SAYISI):
    cerceve = sheet.subsurface(pygame.Rect(i * FRAME_W, 0, FRAME_W, FRAME_H))
    frameler.append(cerceve)

# Animasyon değişkenleri
aktif_frame = 0
animasyon_hizi = 150  # milisaniye
son_güncelleme = pygame.time.get_ticks()
karakter_x, karakter_y = 268, 168

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    # Zamana bagli frame degisimi
    simdi = pygame.time.get_ticks()
    if simdi - son_güncelleme >= animasyon_hizi:
        aktif_frame = (aktif_frame + 1) % FRAME_SAYISI
        son_güncelleme = simdi

    ekran.fill((30, 30, 50))

    # Sprite sheet onizleme (ust kisim)
    ekran.blit(sheet, (44, 20))
    for i in range(FRAME_SAYISI):
        renk = (255, 255, 0) if i == aktif_frame else (100, 100, 100)
        pygame.draw.rect(ekran, renk,
```

```
                            (44 + i * FRAME_W, 20, FRAME_W, FRAME_H), 2)

    # Aktif frame'i buyuk ciz
    buyuk = pygame.transform.scale(frameler[aktif_frame], (128, 128))
    ekran.blit(buyuk, (karakter_x - 64, karakter_y - 64))

    font = pygame.font.SysFont("Arial", 16)
    bilgi = font.render(f"Frame: {aktif_frame} | Hiz: {animasyon_hizi}ms",
                        True, (200, 200, 200))
    ekran.blit(bilgi, (20, 350))

    pygame.display.flip()
    saat.tick(60)

pygame.quit()
```

subsurface Hakkında

`subsurface(Rect)` bir Surface'ın belirli dikdörtgen alanını yeni bir Surface olarak döndürür. Orijinal Surface ile belleği paylaşır, bu yüzden kopyalama maliyeti yoktur. Sprite sheet'ten frame kesmek için en verimli yöntemdir.

Talep 1: Animasyon Hızı Kontrolü

Yukarı/Aşağı ok tuşları ile `animasyon_hizi` değerini 50–500ms aralığında değiştirin. Hız değiştiğinde ekrandaki bilgi metnini güncelleyin.

Talep 2: Yatay Hareket

Sol/Sağ ok tuşları ile karakteri ekranda yatay hareket ettirin. Karakter hareket ederken animasyon çalsın, durduğunda yalnızca ilk frame gösterilsin.

Talep 3: Ping-Pong Animasyon

Frame sırasını 0→1→2→3→2→1→0 şeklinde ileri-geri döndürün. Bir `yon` değişkeni ile frame indeksinin artma veya azalma yönünü takip edin.

2 Çalışma 2: AnimatedSprite Sınıfı

Örnek Kod

Aşağıdaki kodu `animated_sprite.py` dosyasına yazın:

```
import pygame

pygame.init()
ekran = pygame.display.set_mode((600, 400))
pygame.display.set_caption("AnimatedSprite Sınıfı")
saat = pygame.time.Clock()

def sheet_olustur(renkler, boyut=48):
    """Her durum için farkli renkli frame'ler oluşturur."""
    frameler = {}
    for durum, renk_listesi in renkler.items():
        durum_frameleri = []
```

```python
        for renk in renk_listesi:
            yuzey = pygame.Surface((boyut, boyut))
            yuzey.fill((0, 0, 0))
            yuzey.set_colorkey((0, 0, 0))
            pygame.draw.rect(yuzey, renk, (4, 4, boyut-8, boyut-8),
                             border_radius=6)
            durum_frameleri.append(yuzey)
        frameler[durum] = durum_frameleri
    return frameler

class AnimatedSprite(pygame.sprite.Sprite):
    def __init__(self, x, y, animasyonlar):
        super().__init__()
        self.animasyonlar = animasyonlar
        self.durum = "idle"
        self.frame_index = 0
        self.animasyon_hizi = 150
        self.son_güncelleme = pygame.time.get_ticks()
        self.image = self.animasyonlar[self.durum][0]
        self.rect = self.image.get_rect(center=(x, y))

    def durum_değiştir(self, yeni_durum):
        if yeni_durum != self.durum:
            if yeni_durum in self.animasyonlar:
                self.durum = yeni_durum
                self.frame_index = 0
                self.son_güncelleme = pygame.time.get_ticks()

    def update(self):
        simdi = pygame.time.get_ticks()
        if simdi - self.son_güncelleme >= self.animasyon_hizi:
            frameler = self.animasyonlar[self.durum]
            self.frame_index = (self.frame_index + 1) % len(frameler)
            self.image = frameler[self.frame_index]
            self.son_güncelleme = simdi

# Durum renkleri tanımla
renkler = {
    "idle":   [(80, 80, 200), (100, 100, 220), (80, 80, 200)],
    "walk":   [(80, 200, 80), (100, 220, 100),
               (120, 240, 120), (100, 220, 100)],
    "attack": [(200, 80, 80), (240, 120, 60), (255, 60, 60)],
}

animasyonlar = sheet_olustur(renkler)
oyuncu = AnimatedSprite(300, 200, animasyonlar)
grup = pygame.sprite.Group(oyuncu)
font = pygame.font.SysFont("Arial", 18)

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False
        elif olay.type == pygame.KEYDOWN:
            if olay.key == pygame.K_1:
                oyuncu.durum_değiştir("idle")
```

```
            elif olay.key == pygame.K_2:
                oyuncu.durum_değiştir("walk")
            elif olay.key == pygame.K_3:
                oyuncu.durum_değiştir("attack")

    grup.update()

    ekran.fill((30, 30, 50))
    grup.draw(ekran)
    bilgi = font.render(
        f"Durum: {oyuncu.durum} | Frame: {oyuncu.frame_index} "
        f"| 1: idle  2: walk  3: attack",
        True, (200, 200, 200))
    ekran.blit(bilgi, (20, 20))

    pygame.display.flip()
    saat.tick(60)

pygame.quit()
```

State Machine Mantığı

Animasyon durumları arasında geçiş yaparken `frame_index` sıfırlanmalıdır. Aksi halde yeni durumun frame listesi daha kısa ise `IndexError` hatası alırsın. `durum_değiştir()` metodu bu kontrolü otomatik yapar.

Talep 1: Yeni Durum Ekleme

`"death"` adında yeni bir durum ekleyin (3 frame, kırmızıdan siyaha kararma). `4` tuşuna basıldığında bu durum aktif olsun. Death animasyonu sona ulaştığında son frame'de kalsın (döngü yapmasın).

Talep 2: Klavye ile Hareket ve Otomatik Durum

Ok tuşları ile sprite'ı hareket ettirin. Hareket ederken otomatik olarak `"walk"` durumuna, durduğunda `"idle"` durumuna geçsin. `SPACE` tuşuna basıldığında `"attack"` durumuna geçsin.

Talep 3: Durum Bazlı Hız

Her durumun kendi animasyon hızını destekleyin. `animasyon_hizlari` sözlüğü ekleyin: idle→300ms, walk→100ms, attack→80ms. `durum_değiştir()` metodunda hızı da güncelleyin.

3 Çalışma 3: Zamanlayıcılar

Örnek Kod

Aşağıdaki kodu `zamanlayici.py` dosyasına yazın:

```
import pygame

pygame.init()
ekran = pygame.display.set_mode((600, 400))
pygame.display.set_caption("Zamanlayicilar")
```

```python
saat = pygame.time.Clock()
font = pygame.font.SysFont("Arial", 16)

# Cooldown sınıfı
class Cooldown:
    def __init__(self, sure_ms):
        self.sure = sure_ms
        self.son_kullanim = 0

    def hazir_mi(self):
        return pygame.time.get_ticks() - self.son_kullanim >= self.sure

    def kullan(self):
        if self.hazir_mi():
            self.son_kullanim = pygame.time.get_ticks()
            return True
        return False

    def kalan_sure(self):
        gecen = pygame.time.get_ticks() - self.son_kullanim
        kalan = max(0, self.sure - gecen)
        return kalan

# USEREVENT tabanli zamanlayici
DUSMAN_OLAYI = pygame.USEREVENT + 1
pygame.time.set_timer(DUSMAN_OLAYI, 2000)  # 2 saniyede bir

# Cooldown'lar
mermi_cd = Cooldown(500)    # 500ms mermi cooldown
kalkan_cd = Cooldown(3000)  # 3s kalkan cooldown

# Oyun durumlari
mermiler = []
kalkan_aktif = False
kalkan_bitis = 0
dusman_sayaci = 0
oyuncu_x = 300

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False
        elif olay.type == DUSMAN_OLAYI:
            dusman_sayaci += 1
        elif olay.type == pygame.KEYDOWN:
            if olay.key == pygame.K_SPACE:
                if mermi_cd.kullan():
                    mermiler.append([oyuncu_x, 340])
            elif olay.key == pygame.K_s:
                if kalkan_cd.kullan():
                    kalkan_aktif = True
                    kalkan_bitis = pygame.time.get_ticks() + 1500

    # Ok tuslari ile hareket
    tuslar = pygame.key.get_pressed()
    if tuslar[pygame.K_LEFT]:  oyuncu_x -= 4
```

```python
    if tuslar[pygame.K_RIGHT]: oyuncu_x += 4
    oyuncu_x = max(20, min(580, oyuncu_x))

    # Mermi güncelleme
    for m in mermiler:
        m[1] -= 6
    mermiler = [m for m in mermiler if m[1] > 0]

    # Kalkan suresi kontrolu
    if kalkan_aktif and pygame.time.get_ticks() >= kalkan_bitis:
        kalkan_aktif = False

    # --- Cizim ---
    ekran.fill((20, 20, 40))

    # Oyuncu
    renk = (100, 200, 255) if kalkan_aktif else (50, 150, 255)
    pygame.draw.rect(ekran, renk, (oyuncu_x - 15, 340, 30, 30))
    if kalkan_aktif:
        pygame.draw.circle(ekran, (100, 200, 255),
                           (oyuncu_x, 355), 25, 2)

    # Mermiler
    for m in mermiler:
        pygame.draw.rect(ekran, (255, 255, 0), (m[0]-2, m[1], 4, 10))

    # Bilgi paneli
    mermi_kalan = mermi_cd.kalan_sure()
    kalkan_kalan = kalkan_cd.kalan_sure()
    satirlar = [
        f"SPACE: Mermi ({mermi_kalan}ms) | S: Kalkan ({kalkan_kalan}ms)",
        f"Dusman dalgasi: {dusman_sayaci} | Mermi: {len(mermiler)}",
        f"Kalkan: {'AKTIF' if kalkan_aktif else 'Hazir' if kalkan_cd.hazir_mi() else 'Bekleniyor'}"
    ]
    for i, satir in enumerate(satirlar):
        ekran.blit(font.render(satir, True, (200, 200, 200)),
                   (20, 10 + i * 22))

    # Cooldown barlari
    # Mermi CD
    pygame.draw.rect(ekran, (60, 60, 80), (20, 80, 200, 12))
    oran = 1.0 - (mermi_kalan / mermi_cd.sure) if mermi_cd.sure > 0 else 1
    pygame.draw.rect(ekran, (255, 255, 0), (20, 80, int(200*oran), 12))

    # Kalkan CD
    pygame.draw.rect(ekran, (60, 60, 80), (20, 96, 200, 12))
    oran_k = 1.0 - (kalkan_kalan / kalkan_cd.sure) if kalkan_cd.sure > 0 else 1
    pygame.draw.rect(ekran, (100, 200, 255), (20, 96, int(200*oran_k), 12))

    pygame.display.flip()
    saat.tick(60)

```

```
pygame.quit()
```

USEREVENT ve set_timer

`pygame.USEREVENT` sabit bir tamsayıdır. Kendi olay türlerinizi tanımlarken `USEREVENT + 1`, `USEREVENT + 2` gibi değerler kullanın. `set_timer(olay, ms)` belirtilen sürede bir olayı olay kuyruğuna otomatik ekler. Süreyi 0 yaparak zamanlayıcıyı durdurabilirsiniz.

Talep 1: Hızlanan Düşman Dalgaları

Düşman olayının periyodunu her 5 dalgada bir azaltın (2000ms → 1500ms → 1000ms, minimum 500ms). `set_timer()` ile periyodu güncelleyin.

Talep 2: Çoklu Cooldown

Üçüncü bir yetenek ekleyin: `D` tuşu ile "Dash" (cooldown: 2000ms). Dash aktifken oyuncu hızı 3 katına çıksın ve 500ms sürsün. Ekrandaki bilgi paneline dash durumunu da ekleyin.

Talep 3: Geri Sayım Zamanlayıcısı

Oyun başlangıcında 3 saniyelik geri sayım ekleyin. Geri sayım boyunca ekranın ortasında büyük "3…2…1…BASLA!" metni gösterin. Geri sayım bitene kadar oyuncu kontrollerini devre dışı bırakın.

4 Çalışma 4: Görsel Efektler

Örnek Kod

Aşağıdaki kodu `gorsel_efektler.py` dosyasına yazın:

```
import pygame, random, math

pygame.init()
GENISLIK, YUKSEKLIK = 600, 400
ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Gorsel Efektler")
saat = pygame.time.Clock()
font = pygame.font.SysFont("Arial", 16)

class Parcacik:
    def __init__(self, x, y, renk, hiz_x, hiz_y, omur):
        self.x, self.y = x, y
        self.renk = renk
        self.hiz_x, self.hiz_y = hiz_x, hiz_y
        self.omur = omur          # toplam omur (ms)
        self.kalan = omur
        self.yaricap = random.randint(2, 5)

    def güncelle(self, dt):
        self.x += self.hiz_x * dt
        self.y += self.hiz_y * dt
        self.hiz_y += 80 * dt   # yercekimi
        self.kalan -= dt * 1000
```

```python
        return self.kalan > 0

    def ciz(self, yuzey):
        oran = max(0, self.kalan / self.omur)
        r = max(1, int(self.yaricap * oran))
        alfa = int(255 * oran)
        renk = (min(255, self.renk[0]),
                min(255, self.renk[1]),
                min(255, self.renk[2]))
        pygame.draw.circle(yuzey, renk, (int(self.x), int(self.y)), r)

def patlama_olustur(x, y, renk=(255, 150, 50), adet=25):
    parcaciklar = []
    for _ in range(adet):
        aci = random.uniform(0, 2 * math.pi)
        hiz = random.uniform(50, 200)
        hx = math.cos(aci) * hiz
        hy = math.sin(aci) * hiz
        r = (min(255, renk[0] + random.randint(-30, 30)),
             min(255, renk[1] + random.randint(-30, 30)),
             min(255, max(0, renk[2] + random.randint(-30, 30))))
        omur = random.randint(300, 800)
        parcaciklar.append(Parcacik(x, y, r, hx, hy, omur))
    return parcaciklar

# Screen shake
shake_suresi = 0
shake_siddet = 0

def shake_baslat(sure_ms=200, siddet=8):
    global shake_suresi, shake_siddet
    shake_suresi = sure_ms
    shake_siddet = siddet

# Oyun nesneleri
parcaciklar = []
kutular = []
for _ in range(5):
    kx = random.randint(50, GENISLIK - 50)
    ky = random.randint(100, YUKSEKLIK - 100)
    kutular.append(pygame.Rect(kx, ky, 30, 30))

çalıştır = True
while çalıştır:
    dt = saat.tick(60) / 1000.0

    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False
        elif olay.type == pygame.MOUSEBUTTONDOWN:
            mx, my = olay.pos
            vuruldu = False
            for kutu in kutular[:]:
                if kutu.collidepoint(mx, my):
                    parcaciklar.extend(
                        patlama_olustur(kutu.centerx, kutu.centery))
                    kutular.remove(kutu)
```

```
                    shake_baslat(150, 6)
                    vuruldu = True
                    break
            if not vuruldu:
                # Kucuk efekt
                parcaciklar.extend(
                    patlama_olustur(mx, my, (100, 100, 255), 8))

    # Parcacik güncelleme
    parcaciklar = [p for p in parcaciklar if p.güncelle(dt)]

    # Kutu yenileme
    if not kutular:
        for _ in range(5):
            kx = random.randint(50, GENISLIK - 50)
            ky = random.randint(100, YUKSEKLIK - 100)
            kutular.append(pygame.Rect(kx, ky, 30, 30))

    # Screen shake güncelle
    offset_x, offset_y = 0, 0
    if shake_suresi > 0:
        shake_suresi -= dt * 1000
        offset_x = random.randint(-shake_siddet, shake_siddet)
        offset_y = random.randint(-shake_siddet, shake_siddet)

    # --- Cizim ---
    ekran.fill((20, 20, 40))
    cizim_yuzey = pygame.Surface((GENISLIK, YUKSEKLIK))
    cizim_yuzey.fill((20, 20, 40))

    # Kutular
    for kutu in kutular:
        pygame.draw.rect(cizim_yuzey, (200, 60, 60), kutu,
                         border_radius=4)
        pygame.draw.rect(cizim_yuzey, (255, 100, 100), kutu, 2,
                         border_radius=4)

    # Parcaciklar
    for p in parcaciklar:
        p.ciz(cizim_yuzey)

    # Bilgi
    bilgi = font.render(
        f"Tiklayarak kutulari patlat! | Parcacik: {len(parcaciklar)} "
        f"| Kutu: {len(kutular)}", True, (200, 200, 200))
    cizim_yuzey.blit(bilgi, (20, 10))

    # Shake offset ile ciz
    ekran.blit(cizim_yuzey, (offset_x, offset_y))
    pygame.display.flip()

pygame.quit()
```

Parçacık Performansı

Her parçacık bir Python nesnesidir ve çok sayıda parçacık performansı düşürebilir. Tipik bir oyunda aynı anda 200–300 parçacığı aşmamaya çalış. `omur` (lifetime) değerini kısa tutarak parçacık sayısını kontrol altında tutabilirsin.

Talep 1: Renk Bazlı Patlamalar

Her kutuya rastgele bir renk atayın (kırmızı, yeşil, mavi, sarı). Kutu patladığında patlama parçacıkları kutunun renginde olsun. Kutuları `dict` ile saklayın: `{"rect": Rect, "renk": tuple}`.

Talep 2: Kıvılcım İzi

Fare hareket ettikçe küçük kıvılcım parçacıkları bıraksın. `MOUSEMOTION` olayını kullanın. Kıvılcım parçacıkları daha küçük (yarıçap 1–3) ve kısa ömürlü (200–400ms) olmalı. Yerçekimi etkisi zayıf olsun.

Talep 3: Zincirleme Patlama

Bir kutu patladığında yakınındaki kutular da (mesafe $<$ 80 piksel) 300ms gecikme ile patlasın. `get_ticks()` ile gecikme zamanlayıcısı oluşturun. Zincirleme patlamalarda screen shake şiddeti artsın.

Bonus: Patlama Dalgası Efekti

Patlama anında genişleyen yarı saydam bir halka (shockwave) efekti ekleyin. Halka zamanla büyüyüp saydamlaşsın. `pygame.draw.circle()` ile çizin, `set_alpha()` ile saydamlık ayarlayın. Halka yarıçapı 0'dan 100 piksele, saydamlık 255'ten 0'a düşsün. Birden fazla patlama aynı anda olabilmeli.

Başarılar!